

# bank-cua

Record-once / replay-many computer use for legacy back-office banking apps  
Design report & executive summary · Surya Prabhav Gurram

## Executive summary

Banks and credit unions run a long tail of legacy applications with no API — the only way in is to drive the UI like a human operator. **bank-cua** lets an LLM figure out a task *once*, compiles that run into a typed, reviewable **capability artifact**, and then **replays it deterministically with no model in the loop** — the cheap, reliable path an AI agent triggers in production.

The model discovers → the artifact becomes a reusable capability → deterministic replay is how the agent invokes it in production.

### What was built (end-to-end vertical slice)

- Discovery** — goal-driven observe→decide→act loop drives a real, live UI.
- Replay** — deterministic, no-LLM execution with stable locators + checkpoints.
- Safety** — allowlist, irreversible-action gating, secret/PII redaction.
- Cross-tenant** — one artifact reused across a rebranded 2nd tenant via a small override.
- Confidence** — multi-run stability signal + draft→approved gate; bounded assisted recovery.
- Artifact** — typed, versioned, parameterised capability contract (JSON).
- Error taxonomy** — business-outcome vs. recoverable vs. hard-failure.
- Escalation** — human takes control of the *same live session* over CDP.
- Agent API + codegen** — callable-by-name HTTP endpoints; artifact → runnable Playwright.

### Proven, not described — from evidence/

| Scenario                                         | Result                                                                                                             |
|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Member lookup, valid member (12345)              | <b>SUCCESS</b> · outputs <code>member_name</code> , <code>savings_balance=421355</code> (cents)                    |
| Member 00000 / 99999                             | <b>BUSINESS_OUTCOME</b> · <code>MEMBER_NOT_FOUND</code> / <code>PERMISSION_DENIED</code> (not a crash)             |
| Unexpected maintenance interstitial              | <b>RECOVERED</b> · auto-dismissed, run still succeeds                                                              |
| Session timeout injected                         | <b>HARD_FAILURE</b> · <code>SESSION_TIMEOUT</code> surfaced w/ screenshot + DOM                                    |
| Sub-account creation (irreversible)              | <b>HANDOFF</b> · paused → operator confirmed on same session → resumed to SUCCESS                                  |
| Same artifact on a 2nd tenant (rebranded labels) | <b>CROSS-TENANT</b> · clean with a ~4-line override; still works via fallback (drift) without one                  |
| Fill silently discarded by the page              | <b>HARD_FAILURE</b> · <code>FILL_NOT_APPLIED</code> — caught by reading the control back, not by a page checkpoint |

|                                                            |                                                                                          |
|------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Deposit over the ceiling / over the dual-control threshold | <b>REFUSED</b> · VALUE_LIMIT_EXCEEDED / DUAL_CONTROL_REQUIRED — before the browser opens |
| Replayed N times                                           | <b>STABILITY</b> · pass-rate signal feeding the draft→approved gate                      |

Design stance

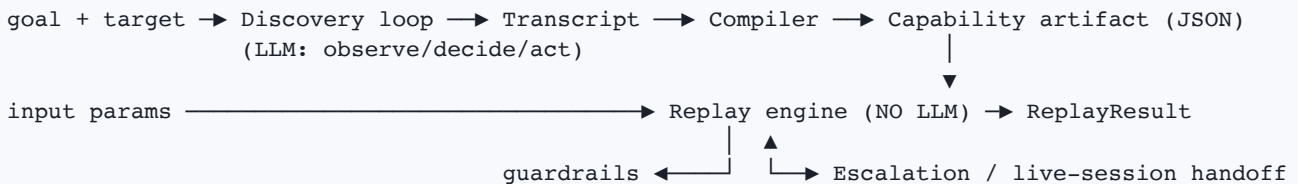
Go deep on the load-bearing pieces — the artifact schema, deterministic replay plus its error contract, and the safety/escalation model — and keep everything else thin at a clean seam. The Surface abstraction and the flow-vs-tenant-binding split in Target keep legacy-web, desktop, and multi-tenant reuse out of the core's way without building that plumbing prematurely. Stack: Python + Playwright, Pydantic v2 schema, a legacy-style Flask mock as the proxy target.

Full source, artifacts, and evidence in the accompanying public repository. 126 automated tests pass; the no-secret sweep of every committed artifact is an enforced test invariant rather than a claim, so it cannot silently regress.

# Design report

## 1 · Architecture

The system is one Python package with sharp internal seams and a single dependency direction: everything speaks the **schema** and the **surface** interface; the only places that speak Playwright are the web surface and the operator's CDP client (the handoff).



- **Surface (perceive/act) vs. the recorded flow.** The schema, discovery loop, and replay engine import only this interface. A legacy-web frame driver or a desktop accessibility driver is a new `Surface` and touches nothing else.
- **Discovery and replay share the same targeting primitives.** Discovery synthesises a `Locator` from the element it chose and acts *through* it, so what is recorded is exactly what is exercised — it already worked once.
- **The model never writes selectors.** It picks elements by an integer `ref`; the system computes the robust, multi-strategy `Locator`. `Locator` quality is a property of code, not prompt luck.
- **The compiler is the record→capability boundary** — parameterising, synthesising checkpoints, attaching the vendor condition library, referencing (not inlining) a redacted transcript.
- **Single process, synchronous, file-backed.** No queue, no services; the parts that would scale (artifact-as-data, catalog, handoff inbox) are already serialisable and stateless. Simplicity is deliberate.

## 2 · Artifact schema

A `CapabilityArtifact` is a **contract**, not a step dump: a calling agent should understand what it needs and returns without reading the steps.

- **Contract-first** — typed `inputs` (with `required`, `sensitive`, `example`), typed `outputs`, and a top-level `success` checkpoint. The catalog turns these into an agent function-calling manifest.
- **Robust targeting as data** — every control is a frame-aware `Locator` with an **ordered candidate list, most-stable-first** (role+name → label → placeholder → text → CSS → XPath), each with human `reasoning`. Read-only values get **label-relative XPath** (`//tr[td="Savings"]/td[last()]`) that survives table reflow.
- **The error taxonomy lives in the schema** — `KnownCondition` records a detector, a class, and an optional recovery, as reviewable data.
- **Safety in the schema** — each step carries a `RiskClass` and `requires_confirmation`; the compiler flags create/confirm/submit as risky.
- **Reuse-ready target** — `Target` separates the shared flow from the per-tenant binding (`base_url`, `tenant_id`, `vendor_product`, `version`).

- **Provenance, decoupled from the transcript** — semver, approval state, `recorded_by`, and a transcript reference + sha256; the raw transcript is stored separately and redacted, never inlined.

### 3 · Determinism & error handling

---

**Determinism.** Replay runs with no model: it resolves each control via the ordered candidates, waits for declared conditions instead of sleeping, and asserts a **per-step checkpoint** after acting before proceeding. Parameters are substituted into URLs, values, and checkpoints at run time.

**The runtime-error contract is the core.** After every step (and on any action failure) replay runs the artifact's condition detectors and acts on the first match:

- `business_outcome` stop and return a code the caller wants — "no such member" / "permission denied" are *answers*, never crashes. A condition may also declare `surfaces_outputs`: Corebank's denial screen withholds the balance but still names the member, which is what the caller needs to route the request, so that declared output is returned alongside the code (listed in `outputs_surfaced`, so partial data never reads as success).
- `recoverable` run the declared recovery (dismiss interstitial, reload transient 500) up to its budget, then re-scan and continue; each recovery is recorded.
- `hard_failure` stop with a structured error: step, expected, observed, plus screenshot + redacted DOM snapshot.

**Declared outputs are guaranteed.** A run cannot report `success` while an output the contract promised is missing — that would be a silent breach, worse than an error because nothing downstream knows to check. Note the split of responsibility: a failed extract is deliberately *not* fatal at the step, so the condition detectors still get their chance to explain *why* the value was absent (`PERMISSION_DENIED` is a better answer to the caller than "extraction failed"); the contract is then enforced once, at the end of the run, as `OUTPUT_EXTRACTION_FAILED` with the blamed step, expected/observed, and full evidence.

**A fill is verified on the control, not the page.** A fill has no page-state consequence, so no checkpoint can see one that silently failed — a readonly, disabled or JS-masked input accepts the keystrokes, the action reports success, and the flow runs on empty data. Fill steps carry `verify_value`: read the control back and assert the write landed (`FILL_NOT_APPLIED`, with evidence). A sensitive value is asserted *non-empty only* — never compared, never logged, because reading a credential back to diff it would reintroduce exactly the leak the observation indexer avoids.

A checkpoint that fails with no matching condition is itself a hard failure with full evidence — unrecognised trouble never silently proceeds. UI drift (secondary) is absorbed by the ordered locator fallbacks and by checkpoints failing loudly; a stability signal and an approval gate are the hooks to catch drift before unattended production.

### 4 · Heterogeneity & multi-tenant

---

**Other surfaces.** The seam is `Surface`. The schema speaks roles, names, labels, and frames — vocabulary a desktop **accessibility tree** exposes just as a browser does — so the format assumes no DOM. Legacy web is the same surface leaning on `frame_path` and structural fallbacks; desktop is a new `Surface` over an OS accessibility API or screenshot+coordinates (the schema already has a coordinate locator kind). Perception/action change; the flow, compiler, replay, error contract, and safety do not.

**Multi-tenant reuse.** `Target` splits the shared flow from the per-tenant binding, so hundreds of tenants on one vendor product share a single artifact and differ only by binding + optional overrides. Two mechanisms make this real: **vendor-level condition libraries** (how "Corebank" signals a locked account is curated once and inherited by all its tenants), and **semantic-first locators** that survive per-tenant theming better than structural paths. Drift is detected by watching checkpoint failures and which locator candidate resolved (both logged); a per-tenant stability score gates unattended use. Canonicalising routes ( `/item/12345` → `/item/:id` ) extends the existing URL parameteriser. **Built and demonstrated:** a rebranded second tenant variant ( `MOCKBANK_VARIANT=submit` , "Member Number"/"Find"/"Log In"); the member-lookup artifact recorded on `demo-cu` runs against `submit-cu` — clean with a ~4-line override, and still succeeding via structural fallbacks (with drift signals) without one. Only the multi-tenant *plumbing* (registry, scheduling) is left unbuilt, per the brief.

## 5 · Escalation & handoff

---

**Detecting "stuck."** Discovery escalates on no-progress over N steps, an unresolvable target, a policy block, or a model `escalate` . Replay escalates on a policy-gated irreversible step or an unrecoverable condition.

**Routing with context.** An `InterventionRequest` carries the capability/goal, current step, live URL, screenshot + DOM, reason, a **control token** ( `agent / operator` ), and the **CDP endpoint** of the live session, to a file-backed inbox; the automation blocks on it (and, unattended, times out and aborts cleanly, preserving the request for triage).

**Taking control of the same live session.** The browser is launched with a remote-debugging port, so the operator attaches to the **same Chromium** via `connect_over_cdp` and drives the **same page** — cookies, half-filled forms, and URL all preserved. Human actions are recorded; on resolve the token returns to `agent` ; replay resumes, or skips execution if the human performed the gated step (so an irreversible action is never double-executed). The full round trip runs in evidence and via the `operator` CLI. The control-transfer *model* is real; the operator *console* is a CLI stand-in for a co-browsing UI, which is out of scope.

## 6 · Safety

---

Guardrails run on **every action, before it happens, in both discovery and replay**; a violation raises, never warns-and-continues.

- **Layered allowlist** — a global policy (URL patterns, action types, an explicit denylist) plus the artifact's own allowlist; a URL must satisfy both, else it is blocked (fail closed).
- **Irreversibility, treated conservatively** — reversible actions are safe; create/confirm/submit/delete are risky and **blocked unless explicitly approved**, or gated behind human confirmation (an escalation when unattended). Refusing to create a sub-account is far cheaper than creating one by mistake.
- **Risk classification — heuristic proposes, human ratifies** — two layered signals: *lexical* (caption matches create/confirm/submit/delete) and *structural* (the control submits a POST form — evidence about what the action *does* rather than what it is called, which survives relabelling and ports to a desktop accessibility tree). Structural corroborates rather than overrides: a POST submit is risky unless its caption is on a short benign list, because promoting every POST would flag "Sign On" and "Search", and a guardrail that cries wolf gets switched off. What it buys is the unlabelled mutation — "Apply", "Proceed" carry no keyword but still change state. The class and its *reason* are persisted, and `catalog`

approve refuses while any risky step is unreviewed; a reviewer may downgrade a false positive with a recorded justification.

- **Value-level (semantic) policy** — the allowlist answers "may the agent click here", never "is this amount sane". Per-parameter rules read the invocation's *inputs* before the browser opens: `max` is a hard ceiling, refused outright and never escalated; `dual_control_above` requires an independent second approver or an operator counter-signature, and fails closed unattended. A run cannot approve itself, and a governed value that will not parse as a number is refused — a limit you cannot evaluate is not a limit. The value gate and the irreversible-step gate are independent and compose.
- **Never persist secrets or regulated data** — sensitive inputs are stored as parameter *names*, never values; redaction is name-based + pattern-based (SSN, card, email) across logs, transcripts, and DOM snapshots. The observation indexer never captures a control's typed value, so secrets never reach the model prompt in the first place. The compiler additionally scrubs run-specific values out of the human-readable step intents — the model narrates in prose ("the provided operator credentials"), which would otherwise bake one run's data, and potentially a real credential, into a committed artifact; inputs become `{param}` placeholders and extracted values `<output>`, in both raw and transformed form. Card detection is Luhn-checked so the safety net does not fire on ordinary digit runs such as timestamps, literal matching is token-bounded, and the mapping form names the parameter (`***REDACTED:username***`) so an over-redaction reads as intentional rather than as corrupted evidence — over-redaction remains the deliberate bias. The repo-wide sweep of `capabilities/` is an **enforced invariant** (`tests/test_artifact_hygiene.py`), not a claim.

**Limits.** Screenshots can still show on-screen PII (treated as sensitive evidence today; masked capture next). Value rules are per-parameter and stateless — they bound one invocation, not a velocity, so ten \$999 deposits in a minute pass every check; aggregate and rate limits are the next layer. The benign-submit list is a curated regex, short and reviewable by design but still maintained per vendor. Dual control is enforced against an honour-system identifier; production would bind it to authenticated operator identity.

## 7 • Cuts

---

Most stretch goals are **built**: agent-facing API (`serve`), code generation (`codegen`), confidence/approval (stability signal + draft→approved gate), bounded assisted recovery, and cross-tenant reuse with per-variant overrides + route canonicalisation. Left thin at clean seams: the **operator console** (CLI stand-in; the handoff mechanism is real); **desktop/legacy-web surfaces** (designed, not built — the `Surface` seam keeps the core uncommitted); **multi-tenant plumbing** (registry/scheduling — the scaling infrastructure the brief says to avoid); and the **committed discovery evidence is a genuine live run** via the Anthropic Messages API (`recorded_by: anthropic`), with a key-free offline reproduction via the bridge provider.

**Next, in order:** (1) a second `Surface` (desktop accessibility tree) to prove the seam on a genuinely non-DOM target; (2) value-level semantic policy (amount limits, dual control on money movement); (3) a real co-browsing console over the existing CDP seam; (4) an artifact auto-repair loop that proposes locator/override updates when drift crosses a threshold, gated by the approval workflow.